

CS3611 Project 9 — 设计文档

1. 文档目的

本文档描述系统的总体架构、模块划分、数据流与 DDoS 攻击流量统计特征定义，为开发、整合、测试提供设计依据。

2. 设计目标

目标	说明
攻击模拟	模拟 SYN Flood、HTTP Flood、UDP 反射放大
流量采集	使用 Wireshark 或 tcpdump 抓包，记录 PPS、协议分布、IP 熵等统计量
智能检测	从 PCAP 提取特征，MLP 模型输出攻击判定 JSON
自动防御	iptables 限速 + 黑名单/loopback 限速，模型决策驱动封禁
效果对比	普通演示：防御前后重复同样攻击，量化流量与负载变化
实时联动	实时演示：混合攻击进行中，后台按窗口循环抓包、推理并自动封禁

安全约束： 所有攻击脚本仅允许目标为 127.0.0.0/8、10.0.0.0/8、172.16.0.0/12、192.168.0.0/16，禁止公网目标。

3. 系统架构图

3.1 模块分层

层次	目录	主要组件
整合层	scripts/	run_demo.sh (普通演示)、run_realtime_demo.sh (实时攻防演示)
攻击层	attacks/	normal_traffic.py、syn_flood.py、http_flood.py、 udp_reflection_sim.py、run_mixed_attack.sh

层次	目录	主要组件
采集层	—	tcpdump、demo_site HTTP 服务 (127.0.0.1:8080)
特征层	features/	extract_features.py → 特征 CSV
模型层	models/	infer.py (MLP 推理) → decision.json
防御层	defense/	iptables_rules.sh、apply_decision.py、block_ip.sh、unblock_all.sh
存储层	storage/	redis_store.py (STORAGE_BACKEND=redis 时双写)

3.2 普通演示 (run_demo.sh)

设计定位： 攻击前抓包 → 离线 MLP 推理与封禁 → 防御后再次攻击，用于展示防御前后流量对比。

数据流：

```
run_demo.sh
├─▶ normal_traffic.py / run_mixed_attack.sh ──▶ demo_site (127.0.0.1:8080)
|
|       run_mixed_attack.sh ──▶ syn_flood + http_flood + udp_reflection (并行)
├─▶ tcpdump ──▶ extract_features.py ──▶ 特征 CSV ──▶ infer.py ──▶ decision.json
├─▶ decision.json ──▶ apply_decision.py ──▶ block_ip.sh ──▶ iptables 规则
├─▶ iptables_rules.sh (基础限速)
├─▶ storage/redis_store.py (启用 Redis 时双写)
└─▶ unblock_all.sh (演示前清理旧规则)
```

时序：

步骤	动作	产出
0	启动受害 HTTP 服务; unblock_all 清空旧规则	服务就绪

步骤	动作	产出
阶段 A	正常流量 10s + 抓包 → 特征提取	normal.pcap/csv
阶段 B	混合攻击 20s + 抓包 → 特征提取 → MLP 推理 → 安装防御	attack_before.*、 decision.json
阶段 C	同参混合攻击 20s + 抓包 → 特征提取	attack_after.*
收尾	输出 PCAP / CSV / 日志路径	演示完成

3.3 实时攻防演示 (run_realtime_demo.sh)

设计定位：混合攻击在前台持续进行时，后台按时间窗口同步完成抓包、特征提取、MLP 推理与自动封禁，用于展示 AI 推理与 iptables 的在线联动。

数据流：

```
run_realtime_demo.sh
├─▶ 正常流量基线 + 抓包（可选对照）
├─▶ 前台：run_mixed_attack.sh（SYN+HTTP+UDP 持续运行，默认 120s）
└─▶ 后台：realtime_defense_loop（每 REALTIME_WINDOW_SECONDS 秒一轮）
    ├─▶ tcpdump 抓窗口 PCAP
    ├─▶ extract_features.py → 窗口特征 CSV
    ├─▶ infer.py → decision_window*.json
    ├─▶ apply_decision.py → block_ip.sh → iptables 规则
    ├─▶ 合并为 decision_realtime_<run_id>.json
    └─▶ storage/redis_store.py（启用 Redis 时写入 defense_actions 等）
```

时序：

步骤	执行方	动作	产出
1	run_realtime_demo.sh	启动受害服务；正常流量基线 + 抓包	normal_<run_id>.pcap/csv
2	前台	run_mixed_attack.sh 持续攻击	攻击日志

步骤	执行方	动作	产出
3	后台	realtime_defense_loop 每 4s 一轮	window_* pcap/csv/decision
4	后台	apply_decision.py 自动封禁	iptables 规则、 defense_blocks.log
5	脚本收尾	合并窗口决策；输出摘要	realtime_window_summary_*.csv、 decision_realtime_*.json

与普通演示的区别：

对比项	普通演示	实时攻防演示
封禁时机	攻击结束后离线推理，再复测防御后流量	攻击进行中按窗口在线推理并封禁
决策粒度	单次 decision.json	多窗口 decision_window_*.json 合并
效果展示	防御前/后 PCAP 与 PPS 对比	流量时间线 + 窗口级封禁流水线
入口脚本	run_demo.sh	run_realtime_demo.sh

4. 模块设计说明

4.1 攻击模块 (attacks/)

组件	功能
normal_traffic.py	模拟正常 HTTP 请求
syn_flood.py	TCP SYN 洪泛，伪造 127.x 源 IP
http_flood.py	高频率 GET/POST，记录 HTTP 成功率
udp_reflection_sim.py	模拟 UDP 反射放大流量
run_mixed_attack.sh	并行启动 SYN+HTTP+UDP 混合攻击

4.2 防御模块 (defense/)

组件	功能	对应必做要求
iptables_rules.sh	SYN≤50/s、HTTP≤120/s、异常 TCP 丢弃	流量限速
block_ip.sh	私网源 DROP; 127.x hashlimit 限速	IP 黑名单
nftables_rules.sh	可选, 过滤非法协议包	流量清洗
apply_decision.py	读取 decision.json 自动封禁	自动化防御
nginx.conf	可选 CDN 反代 (扩展)	CDN 集成

4.3 特征与模型模块

组件	功能
features/extract_features.py	PCAP → 统计特征 CSV
models/train_mlp.py / infer.py	MLP 训练与推理
models/anomaly_kmeans.py / anomaly_autoencoder.py	无监督异常检测 (扩展, 独立脚本)

4.4 整合入口

脚本	用途
scripts/run_demo.sh	普通演示: 攻击前 → 推理封禁 → 攻击后对比
scripts/run_realtime_demo.sh	实时演示: 攻击进行中后台窗口推理与封禁
scripts/visualize_demo.py	普通演示 HTML 仪表盘
scripts/visualize_realtime_demo.py	实时演示 HTML 仪表盘

运行步骤见 report/操作手册.md。

5. DDoS 攻击流量统计特征表

5.1 核心特征

字段	物理含义	正常流量	攻击流量
pps	每秒包数，反映洪泛强度	低 (~60)	高 (峰值 1000+)
bps	每秒比特数，反映带宽占用	低	高
syn_count	SYN 包数量	低	SYN Flood 时极高
syn_ack_ratio	SYN/ACK 比值，衡量连接异常	正常比例	显著升高
unique_src_ips	窗口内不同源 IP 数	1	数十 (伪造多源)
ip_entropy	源 IP 分布熵	低	高
avg_pkt_size	平均包长 (字节)	较稳定	随协议变化
ack_count	ACK 包数量	相对较高	相对 SYN 偏低

辅助字段：src_ip (供防御封禁)、label / attack_type (训练标注)。完整定义见 features/feature_schema.md。

5.2 普通演示实测 (Run report_demo_01)

场景	峰值 PPS	说明
正常流量	60	基线
攻击 (防御前)	1475	混合攻击洪泛高峰
攻击 (防御后)	411	智能防御后峰值降约 72%

5.3 实时演示实测 (Run realtime_demo_01)

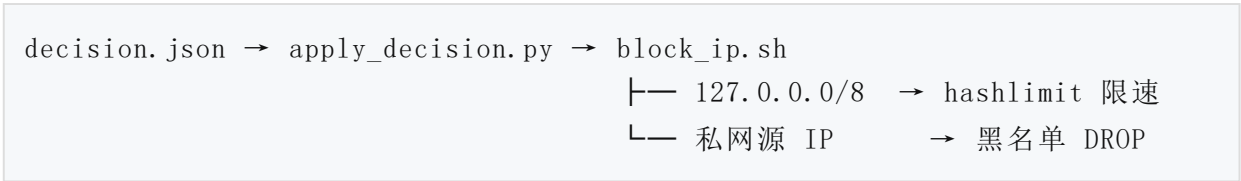
指标	数值
正常基线峰值	60 PPS
攻防峰值	2.0K PPS
实时窗口数	3 (extract/infer/apply 均为 ok)
窗口决策总数	616
最高置信度	1.000
首封延迟	约 46 s

6. 模型决策与防御策略

models/infer.py 输出 decision.json, 核心字段: src_ip、confidence、action (block)、reason。

实时演示合并文件 decision_realtime_<run_id>.json 额外包含 mode: "realtime" 与 window 字段。

模型驱动封禁:



基础限速: SYN ≤ 50/s, HTTP ≤ 120/s, 与模型封禁并行生效。普通演示与实时演示共用同一套 apply_decision.py + block_ip.sh 链路。

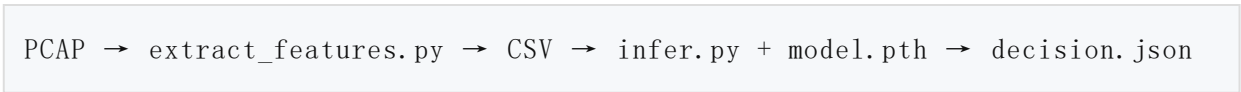
7. 扩展功能设计

7.1 扩展功能总览

扩展项	实现模块
智能攻击分类器	features/ + train_mlp.py + infer.py
无监督异常检测	anomaly_kmeans.py + anomaly_autoencoder.py
自动化防御机制	apply_decision.py + block_ip.sh
CDN 集成	defense/nginx.conf
Redis 结构化存储	storage/redis_store.py

7.2 智能攻击分类器

从 PCAP 提取 PPS、SYN/ACK 比、IP 熵等统计特征, PyTorch MLP 二分类输出 decision.json; 推理阈值默认 0.80。



7.3 无监督异常检测

K-Means 对高 PPS / 高 SYN/ACK 簇判异常；AutoEncoder 以重构误差判异常。输入同 MLP 特征 CSV，输出异常报告与模型文件，不驱动 `apply_decision`。

7.4 自动化防御机制

`infer.py` → `apply_decision.py` → `block_ip.sh` → `iptables` + `defense_blocks.log`，实现检测→阻断闭环。

7.5 CDN 集成

`defense/nginx.conf`: 8081 反代 8080，含 `limit_req_zone` 边缘限速与 `proxy_cache_path` 缓存。默认演示直连源站，CDN 手动启用。

7.6 Redis 结构化存储

`STORAGE_BACKEND=redis` 时将特征、决策、封禁动作双写 Redis Stream；文件产物 (CSV/JSON/PCAP) 仍为合同主输出。详见 `docs/storage.md`。
